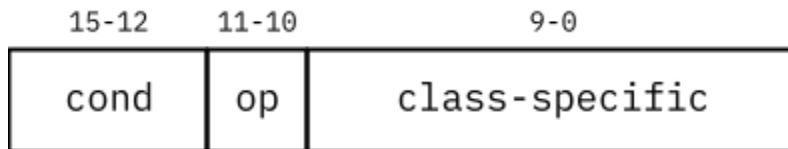


ISA Documentation

Overview

- Word-addressed, 16-bit data path
- Address space bounded by 16-bit arithmetic (64K of word space)
- Every instruction has a 4-bit condition field, similar to ARM
- Four different instruction classes, including **Data Processing**, **Memory**, **Branch**, and **Special**



op field	class
00	Data Processing
01	Memory
10	Branch
11	Special

Assembly Syntax

General form: **[instr] [cond] [operand] [operand] [operand]**

- **instr** is a mnemonic for a command (ADD, LDR, B, HALT, etc.).
- Blank lines and lines beginning with **//** are ignored.
 - Comments cannot be on the same line as instructions, even if **//** is used.
- A line consisting of a single word (not a mnemonic) creates a **label** at the current instruction address, to be used with Branch instructions as a target.
- Every instruction has an explicit condition (**cond**) mnemonic. Unconditional is not declared by default, but must instead be written as **AL** similar to every other condition.
 - The condition field used is the same as the one used by **ARM** assembly.
- Examples:
 - **ADD AL R0 R1**
 - **CSP LS R4**
 - **STR GT R0 R3 R1**
 - **branchLink**
 - **BL EQ branchLink**
 - **POP AL R2**
 - **HALT LE**

Label and Branch Offsets

Single-word labels that do not appear in any instruction mnemonics are defined as labels in the program. However, they are replaced by the condition **AL NOP (1110110100000000)**, using up a full instruction.

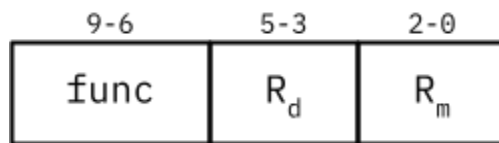
For **B/BL** instruction referencing a label, the assembler will point to the instruction after the label's address, or an offset of the branch instruction's address minus the label's address

plus one. Because of bitwidth limitations, **B/BL** instructions can only point to targets within -256 to 255 addresses away from the **B/BL** instruction.

Condition Mnemonics

cond	mnemonic	meaning
0000	EQ	Equal
0001	NE	Not equal
0010	CS	Unsigned $\geq$
0011	CC	Unsigned $<$
0100	MI	Negative
0101	PL	Positive or zero
0110	VS	Overflow
0111	VC	No overflow
1000	HI	Unsigned $>$
1001	LS	Unsigned $\leq$
1010	GE	Signed $\geq$
1011	LT	Signed $<$
1100	GT	Signed $>$
1101	LE	Signed $\leq$
1110	AL	Always

Data Processing



Func selects one of 16 operations. **R_d** and **R_m** each address one of the 8 registers from R0 to R7.

func	mnemonic	operation
0000	ADD	$R_d \leftarrow R_d + R_m$
0001	SUB	$R_d \leftarrow R_d - R_m$
0010	AND	$R_d \leftarrow R_d \& R_m$
0011	ORR	$R_d \leftarrow R_d R_m$
0100	EOR	$R_d \leftarrow R_d \wedge R_m$
0101	MVN	$R_d \leftarrow \sim R_m$
0110	CMP	flags $\leftarrow R_d - R_m$ (result discarded)
0111	LSL	$R_d \leftarrow R_d \ll R_m$
1000	LSR	$R_d \leftarrow R_d \gg R_m$
1001	ASR	$R_d \leftarrow R_d \ggg R_m$
1010	ROR	$R_d \leftarrow R_d$ rotated right by R_m
1011	MOV	$R_d \leftarrow R_m$
1100	ADDS	$R_d \leftarrow R_d + R_m$ (flags updated)
1101	SUBS	$R_d \leftarrow R_d - R_m$ (flags updated)

1110	CSP	Rd ← SP	*
1111	CPO	Rd ← 0x0001	*

Only the commands **ADDS**, **SUBS**, and **CMP** update the flags.

Assembly Syntax

General form: **instr cond R_d R_m**

* - The exceptions to the general form are **CSP** and **CPO**, which take only one register operand (CSP **cond** R_d and CPO **cond** R_d). The assembler fills the unused **R_m** field with zeros (000).

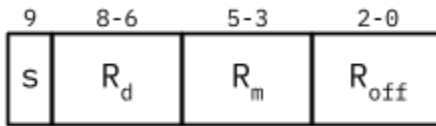
CSP

As one of the two nonstandard functions, **CSP**, or copy SP, is used to access the stack pointer. Because SP is not a general-purpose register file, but instead standalone with dedicated adders for **PUSH/POP**, it is only reachable with the **CSP** command, which copies the value to **R_d**.

CPO

As the other nonstandard function, **CPO**, or copy one, is implemented for bootstrapping purposes. Without an immediate field, this CPU has no way to load nonzero values into registers. The **CPO** function gives access to a nonzero constant from a cold reset where all registers and memory start at zero, and allows the user to build up to any other constant using the **Data Processing** functions.

Memory



s	mnemonic	operation
0	LDR	$R_d \leftarrow \text{MEM}[R_m + R_{off}]$
1	STR	$\text{MEM}[R_m + R_{off}] \leftarrow R_d$

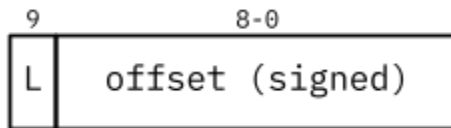
Addressing is register-indexed only, where the address is $R_m + R_{off}$. There is no immediate/offset-literal form. It can also only use general-purpose registers – SP cannot be directly accessed by memory instructions.

Assembly Syntax

General form: **instr cond R_d R_m R_{off}**

There are no exceptions, and all three registers are required for the instruction. There is also no shorthand for a zero offset.

Branch



L	mnemonic	operation
0	B	$PC \leftarrow PC + \text{offset}$
1	BL	$LR \leftarrow PC + 1$ $PC \leftarrow PC + \text{offset}$

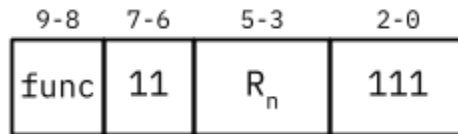
The offset is a signed, PC-relative 9-bit value. Without a dedicated return instruction, returns or indirect jumps are executed using **MOV PC, LR** or any other **Data Processing/Memory** instruction that happens to target R7.

Assembly Syntax

General form: **instr cond label**

Raw numeric-offsets are not supported. Branches always target a label, and the assembler resolves the offset. The label can be any string, as long as it remains a single word with no whitespace in between.

Special



func	mnemonic	operation
00	HALT	Freezes PC and stops the CPU
01	NOP	No operation
10	PUSH	$SP \leftarrow SP - 1$ $MEM[SP] \leftarrow R_n$
11	POP	$R_n \leftarrow MEM[SP]$ $SP \leftarrow SP + 1$

R_n is unused for **HALT** and **NOP**. **PUSH/POP** implement a full descending stack. PUSH decrements SP before writing, POP reads before incrementing SP. At startup, SP is reset to the top of MEM, so the stack grows downward from the top of data memory.

Assembly Syntax

General form (**PUSH/POP**): **instr cond R_n**

General form (**HALT/NOP**): **instr cond**

Register Map

register	role
R0	General-purpose register Return value
R1	General purpose register Additional arguments
R2	General purpose register Additional arguments
R3	General purpose register
R4	General purpose register
R5	General purpose register
R6 (LR)	Link register Written by BL and used for indirect jumps or returns
R7 (PC)	Program counter Part of the ordinary register file and accessible for operations
SP	Stack pointer Standalone register outside the standard 3-bit field with dedicated adders, touched only by PUSH/POP and CSP